# IT STUDY HUB

## COMPLETE C++ PROGRAMMING SERIES

MODULE 7: STL & GENERIC PROGRAMMING

**Authors:** Mithila Rani Saha & Dithsa Dutta

**Website:** www.itstudyhub.dpdns.org

# Module Overview

### 1. About This Module

Welcome to **Module 7: STL & Generic Programming** in the IT Study Hub Complete C++ Programming Series. Building upon the file handling, exception safety, and foundational STL concepts introduced in Module 6, this module dives deeply into the robust architecture of the Standard Template Library (STL). Authored by Mithila Rani Saha and Dithsa Dutta, this

handbook masters sequence and associative containers, iterator categories, high-performance algorithms, lambda expressions, function objects, pairs, tuples, and generic programming principles.

## 2. Learning Outcomes

By completing this module, you will be able to:

- Select and implement the ideal STL sequence (`vector`, `deque`, `list`, `forward_list`) or associative (`set`, `map`, `unordered_set`, `unordered_map`) container based on performance requirements.
- Utilize container adapters (`stack`, `queue`, `priority_queue`) for specialized data organization.
- Traverse containers proficiently using input, output, forward, bidirectional, random-access, `const`, and reverse iterators.
- Apply powerful STL algorithms (`sort`, `find`, `binary_search`, `lower_bound`, `upper_bound`, `accumulate`, `min_element`, `max_element`) with custom lambda functions and function objects.
- Design reusable generic components utilizing function and class templates alongside utility structures (`pair`, `tuple`).

## 3. Module Coverage

This module comprehensively covers:

- **STL Architecture:** Containers, Iterators, and Algorithms.
- **Sequence Containers:** `vector`, `deque`, `list`, `forward_list`.
- **Associative & Unordered Containers:** `set`, `multiset`, `map`, `multimap`, `unordered_set`, `unordered_map`.
- **Container Adapters:** `stack`, `queue`, `priority_queue`.
- **Iterators:** Categories, `const_iterator`, and reverse iterators.
- **Algorithms:** Sorting, searching, binary searching, numerical accumulation, and extrema.

- **Callable Objects:** Lambda functions, function objects (functors), and `std::function`.
- **Utilities:** `pair`, `tuple`, and Generic Programming basics.
- **Hands-On Project:** Advanced Inventory Management System.
- **Rigorous Assessments:** 15+ practice questions, 20+ viva questions with answers, 15+ technical interviews, and Big-O complexity notes.

IT STUDY HUB

# 4. Main Notes: Theoretical Foundations

Generic programming and the Standard Template Library empower C++ developers to write highly reusable, type-safe, and lightning-fast software. Below are the exhaustive foundational notes for Module 7.

## Concept: STL Architecture, Sequence Containers (vector, deque, list, forward_list)

**Simple Explanation:** The STL architecture separates data storage (containers) from data processing (algorithms) using traversal bridges (iterators). Sequence containers store elements in linear sequences: `vector` (dynamic contiguous array), `deque` (double-ended queue), `list` (doubly linked list), and `forward_list` (singly linked list).

> ***Real-Life Analogy:*** *Think of `vector` as a continuous line of parked cars (fast random access), `deque` as a row of train cars accessible from both ends, and `list` as a treasure hunt where each clue holds the address of the next and previous clue (fast insertion/deletion everywhere).*

```
Class Diagram / Architecture:
Containers (vector/list/deque) → Iterators → Algorithms
```

```
Syntax:
#include
#include
#include
vector v;
deque dq;
list lst;
```

```
#include
#include
#include
using namespace std;

int main() {
    vector vec = {10, 20, 30};
    vec.push_back(40);

    list lst = {5, 15, 25};
    lst.push_front(2); // O(1) front insertion in list

    cout << "Vector back: " << vec.back() << endl;
    cout << "List front: " << lst.front() << endl;
    return 0;
}
```

**Output:**

```
Vector back: 40
List front: 2
```

**Line-by-Line Breakdown:**

- **vector<int> vec**: Contiguous memory container supporting O(1) random access.
- **list<int> lst**: Doubly linked list supporting O(1) insertion/deletion at any iterator position.

**Real-World Application:** Managing dynamic task queues in operating system schedulers using `deque` or `list` structures.

**Common Mistakes:** Inserting elements in the middle of a `vector` frequently, which causes expensive element shifting.

**Best Practices:** Use `vector` as your default sequence container due to cache locality unless frequent front/middle insertions are required.

> **Actionable Practice:** Create a `deque` and perform push and pop operations on both its front and back ends.

## Concept: Associative and Unordered Containers (set, map, unordered_set, unordered_map)

**Simple Explanation:** Associative containers (`set`, `map`, `multiset`, `multimap`) store elements in sorted binary search trees (Red-Black Trees), guaranteeing logarithmic time complexity $O(\log N)$ for lookups. Unordered containers use hash tables, offering average constant time $O(1)$ lookups.

> ***Real-Life Analogy:*** *An ordered `map` is like a physical dictionary where words are sorted alphabetically ($O(\log N)$ search). An `unordered_map` is like a digital hash index lookup table where a key instantly hashes directly to its value ($O(1)$).*

```
Class Diagram / Architecture:
set/map (Red-Black Tree: Ordered) | unordered_set/unordered_map
(Hash Table: O(1))
```

```
Syntax:
#include
#include
map orderedMap;
unordered_map hashTable;
```

```
#include
#include
#include
using namespace std;

int main() {
```

```
    map stock;
    stock["Apple"] = 50;
    stock["Banana"] = 120;

    // Automatically sorted by key alphabetically
    cout << "--- Ordered Map ---:\n";
    for(const auto &p : stock) {
        cout << p.first << ": " << p.second << endl;
    }
    return 0;
}
```

**Output:**

```
--- Ordered Map ---:
Apple: 50
Banana: 120
```

**Line-by-Line Breakdown:**

- **map<string, int> stock**: Stores key-value pairs sorted automatically by key.
- Iterating through `stock` prints keys in alphabetical order.

**Real-World Application:** Fast database indexing (`unordered_map`) and sorted symbol tables in compilers (`map`).

**Common Mistakes:** Using `std::map` when order is irrelevant, sacrificing $O(1)$ hash table performance for $O(\log N)$ tree lookups.

**Best Practices:** Default to `unordered_map` for fastest key-value lookups when ordering is not required.

**Actionable Practice:** Create a `map` storing student names and scores, and retrieve a score by name.

## Concept: Container Adapters (stack, queue, priority_queue)

**Simple Explanation:** Container adapters provide restricted, specialized interfaces wrapping around underlying sequence containers. `stack` implements LIFO (Last-In-First-Out), `queue` implements FIFO (First-In-First-Out), and `priority_queue` serves as a max-heap where the largest element always stays on top.

> ***Real-Life Analogy:*** *A `stack` is like a stack of cafeteria plates (you can only take the top plate). A `queue` is like a ticket checkout line (first person in line is served first). A `priority_queue` is like an emergency room triage where critical patients are seen first regardless of arrival time.*

> **Class Diagram / Architecture:**
> **Stack (LIFO) | Queue (FIFO) | Priority Queue (Max-Heap)**

> **Syntax:**
> **#include**
> **#include**
> **stack s;**
> **queue q;**
> **priority_queue pq;**

```
#include
#include
using namespace std;

int main() {
    priority_queue pq;
    pq.push(10);
    pq.push(50);
    pq.push(20);

    cout << "Priority Queue top (Max-Heap): " << pq.top() << endl;
    pq.pop();
```

```
    cout << "New top after pop: " << pq.top() << endl;
    return 0;
}
```

**Output:**
```
Priority Queue top (Max-Heap): 50
New top after pop: 20
```

**Line-by-Line Breakdown:**

- **priority_queue<int> pq**: Automatically organizes elements so the largest is always at `.top()`.
- **pq.pop()**: Removes the highest priority element.

**Real-World Application:** Undo/Redo functionality in text editors (`stack`) and Dijkstra's shortest path graph algorithms (`priority_queue`).

**Common Mistakes:** Attempting to iterate through a stack or queue using iterators (adapters deliberately hide iterators to enforce LIFO/FIFO constraints).

**Best Practices:** Use `priority_queue` with `std::greater` as a comparator when you need a min-heap instead of a max-heap.

**Actionable Practice:** Push 5 numbers into a stack and pop them to verify LIFO order.

## Concept: Iterators (Categories, const_iterator, Reverse iterators)

**Simple Explanation:** Iterators are generalized pointer-like objects that iterate through container elements. Categories include Input, Output, Forward, Bidirectional, and Random-Access iterators.

`const_iterator` prevents modifying elements during traversal, while reverse iterators traverse backward.

> ***Real-Life Analogy:*** *An iterator is like a bookmark in a book. It keeps your exact place on a page and lets you flip forward or backward through chapters without knowing how the book is bound internally.*

> **Class Diagram / Architecture:**
> **Container → `begin()` / `end()` → Iterator traversal → Dereference `*it`**

> **Syntax:**
> **vector::iterator it;**
> **vector::const_iterator cit;**
> **vector::reverse_iterator rit;**

```
#include
#include
using namespace std;

int main() {
    vector v = {1, 2, 3, 4, 5};

    cout << "Reverse traversal: ";
    for(auto rit = v.rbegin(); rit != v.rend(); ++rit) {
        cout << *rit << " ";
    }
    cout << endl;
    return 0;
}
```

> **Output:**
> Reverse traversal: 5 4 3 2 1

**Line-by-Line Breakdown:**

- **v.rbegin()**: Returns a reverse iterator pointing to the last element.
- **++rit**: Moves backward toward the beginning of the vector.

**Real-World Application:** Writing generic algorithms that work identically across vectors, lists, and deques without caring about underlying memory structures.

> **Common Mistakes:** Dereferencing past-the-end iterators (`end()`), which causes immediate undefined behavior and crashes.

> **Best Practices:** Use `const_iterator` (or `cbegin()` / `cend()`) whenever traversal is read-only to guarantee data safety.

> **Actionable Practice:** Traverse a vector of strings forward and backward using iterators.

## Concept: Algorithms (sort, find, binary_search, lower/upper_bound, accumulate, etc.)

**Simple Explanation:** The `` and `` headers provide dozens of optimized generic routines operating on iterator ranges: sorting (`sort`), searching (`find`, `binary_search`), range bounding (`lower_bound`, `upper_bound`), and accumulation (`accumulate`).

> ***Real-Life Analogy:*** *STL algorithms are like heavy-duty factory machinery. You feed them raw material boundaries (begin and end iterators), and they instantly sort, filter, or accumulate data without requiring you to write manual loops.*

```
Class Diagram / Architecture:
Iterator Range `[begin, end)` → STL Algorithm → Computed Result
```

```
Syntax:
#include
#include
sort(v.begin(), v.end());
auto it = find(v.begin(), v.end(), val);
int sum = accumulate(v.begin(), v.end(), 0);
```

```
#include
#include
#include
#include
using namespace std;

int main() {
    vector v = {30, 10, 50, 20, 40};

    sort(v.begin(), v.end());
    int sum = accumulate(v.begin(), v.end(), 0);

    bool found = binary_search(v.begin(), v.end(), 30);

    cout << "Sum: " << sum << ", Found 30: " << (found ? "Yes" :
"No") << endl;
    return 0;
}
```

```
Output:
Sum: 150, Found 30: Yes
```

**Line-by-Line Breakdown:**

- **sort(...)**: Sorts elements in ascending order in $O(N \log N)$ time.
- **accumulate(...)**: Sums up all elements starting from initial value 0.
- **binary_search(...)**: Fast $O(\log N)$ search on sorted ranges.

**Real-World Application:** Financial analytics engines computing sums, averages, and sorting customer records instantly.

**Common Mistakes:** Running `binary_search` or `lower_bound` on an unsorted container (results in undefined behavior).

**Best Practices:** Always ensure container ranges are sorted before invoking binary search algorithms.

**Actionable Practice:** Find the minimum and maximum elements in a vector of floats using `min_element` and `max_element`.

## Concept: Callable Objects: Lambda Functions, Functors, and std::function

**Simple Explanation:** Lambda functions are anonymous inline closures. Function objects (functors) are classes overloading `operator()`. `std::function` is a polymorphic wrapper capable of storing any callable target.

> ***Real-Life Analogy:*** *A lambda function is like a disposable sticky note with a quick instruction written on it, used once and discarded. A functor is like a specialized multi-purpose stamp machine programmed to execute a specific routine whenever pressed.*

```
Class Diagram / Architecture:
Lambda `[](int x){ return x > 5; }` → Inline Callable Object
```

```
Syntax:
auto lambda = [](int a, int b) { return a + b; };
struct Functor { bool operator()(int x) { return x > 0; } };
```

```
#include
#include
#include
using namespace std;

int main() {
    vector v = {1, 2, 3, 4, 5, 6};

    // Using lambda with count_if
    int evenCount = count_if(v.begin(), v.end(), [](int n) {
        return n % 2 == 0;
    });

    cout << "Even count: " << evenCount << endl;
    return 0;
}
```

**Output:**

```
Even count: 3
```

**Line-by-Line Breakdown:**

- **[](int n) { return n % 2 == 0; }**: Anonymous lambda predicate passed directly into `count_if`.

**Real-World Application:** Event-driven GUI programming and asynchronous callback handlers in high-frequency trading systems.

**Common Mistakes:** Capturing local variables by reference `[&]` in async lambdas whose stack frames expire before execution.

**Best Practices:** Capture only required variables explicitly (`[x, &y]`) rather than capturing everything blindly by reference `[&]`.

**Actionable Practice:** Write a lambda function that checks if a string exceeds length 5.

## Concept: Utilities: pair, tuple, and Generic Programming Basics

**Simple Explanation:** `std::pair` bundles two heterogeneous objects, while `std::tuple` generalizes this to an arbitrary number of values. Generic programming relies on templates (`template `) to write code independent of specific data types.

> ***Real-Life Analogy:*** *A `pair` is like a dual-pocket wallet. A `tuple` is like a multi-layered utility briefcase with custom compartments for holding different items of any size.*

**Class Diagram / Architecture:**
**Template `` → Generic Code Generation for int, float, custom classes**

**Syntax:**

```
pair p = {1, "One"};
tuple t(1, "Test", 9.9);

template
T getMax(T a, T b) { return (a > b) ? a : b; }
```

```
#include
#include
using namespace std;

template
T square(T x) {
    return x * x;
}

int main() {
    tuple student(101, "Mithila", 98.5);

    cout << "ID: " << get<0>(student) << endl;
    cout << "Name: " << get<1>(student) << endl;
```

```
    cout << "Square of 5.5: " << square(5.5) << endl;
    return 0;
}
```

```
Output:
ID: 101
Name: Mithila
Square of 5.5: 30.25
```

**Line-by-Line Breakdown:**

- **template <typename T>**: Generates type-safe code dynamically for any data type (`int`, `double`, etc.).
- **tuple<...>**: Holds heterogeneous data retrieved via `get()`.

**Real-World Application:** Writing generic mathematical library functions that operate identically on integers, floats, and complex numbers.

**Common Mistakes:** Using magic indices in `std::tuple` instead of `std::tie` or structured bindings (`auto [id, name, gpa] = student;`).

**Best Practices:** Use modern C++ structured bindings (`auto& [x, y] = myPair;`) to unpack pairs and tuples cleanly.

**Actionable Practice:** Write a generic template function that swaps two variables of any data type.

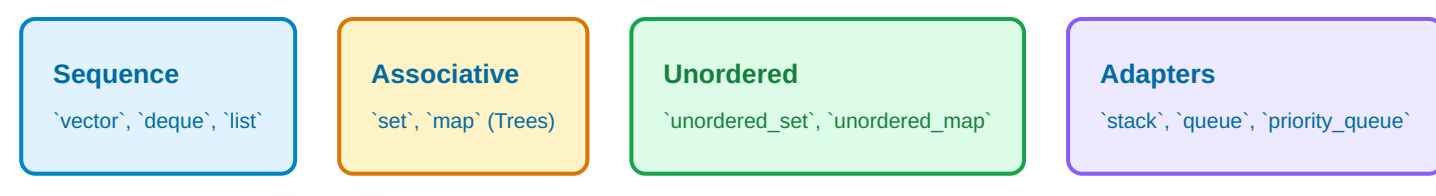
# Visual Workflows & Architecture

Architectural diagrams mapping STL container taxonomies, iterator category hierarchies, lambda execution flow, and generic template compilation.

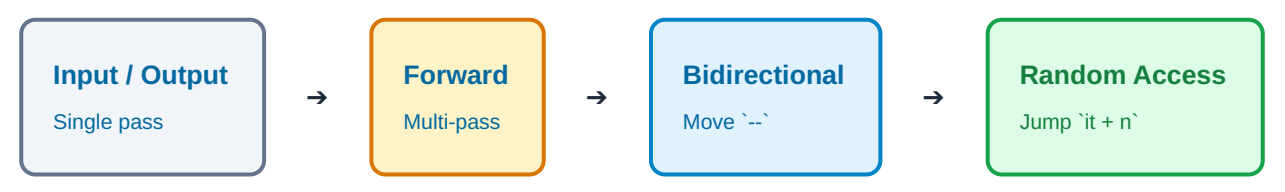
### Diagram 1: STL Architecture (The Three Pillars)



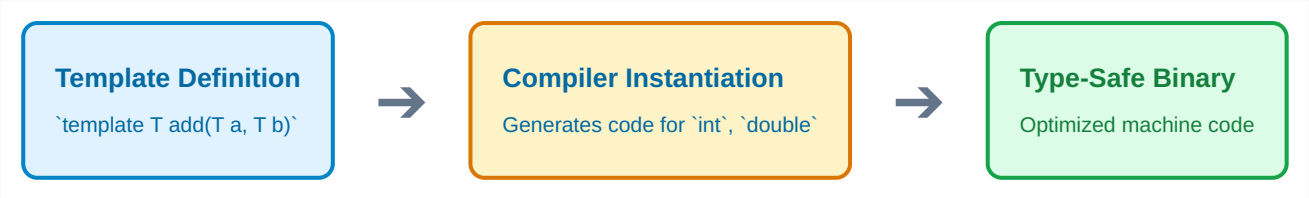

### Diagram 2: Container Categories




### Diagram 3: Iterator Category Hierarchy



## Diagram 4: Generic Programming Template Workflow




IT STUDY HUB

# Comparative Analysis

Professional C++ development requires choosing the precise container, iterator, and algorithm for optimal Big-O complexity.

### Table 1: vector vs deque vs list

| Feature | `std::vector` | `std::deque` | `std::list` |
|---|---|---|---|
| **RAM Layout** | Contiguous buffer | Chunked contiguous blocks | Doubly linked nodes |
| **Random Access** | $O(1)$ (`[]`) | $O(1)$ (`[]`) | $O(N)$ (no random access) |
| **Front Insert/ Delete** | $O(N)$ (expensive shift) | $O(1)$ | $O(1)$ |

### Table 2: set vs multiset and map vs multimap

| Container | Unique Keys? | Ordered? | Underlying Structure |
|---|---|---|---|
| **`set`** | Yes | Yes | Red-Black Tree ($O(\log N)$) |
| **`multiset`** | No (duplicates allowed) | Yes | Red-Black Tree ($O(\log N)$) |
| **`map`** | Yes | Yes | Red-Black Tree ($O(\log N)$) |
| **`multimap`** | No (duplicate keys allowed) | Yes | Red-Black Tree ($O(\log N)$) |

### Table 3: map vs unordered_map

| Aspect | `std::map` | `std::unordered_map` |
|---|---|---|
| **Ordering** | Sorted by key | Unordered |
| **Search / Insert Complexity** | $O(\log N)$ | $O(1)$ average ($O(N)$ worst case hash collision) |
| **Underlying Structure** | Red-Black BST | Hash Table with Buckets |

### Table 4: stack vs queue vs priority_queue

| Adapter | Discipline | Primary Methods |
|---|---|---|
| **`stack`** | LIFO (Last-In-First-Out) | `push()`, `pop()`, `top()` |
| **`queue`** | FIFO (First-In-First-Out) | `push()`, `pop()`, `front()`, `back()` |
| **`priority_queue`** | Max-Heap / Min-Heap | `push()`, `pop()`, `top()` |

### Table 5: iterator vs const_iterator

| Aspect | `iterator` | `const_iterator` |
|---|---|---|
| **Element Modification** | Can modify container elements (`*it = val`) | Read-only access (cannot modify elements) |
| **Iterator Movement** | Can increment/decrement (`++it`, `--it`) | Can increment/decrement (`++it`, `--it`) |

### Table 6: sort vs stable_sort and find vs binary_search

| Algorithm Pair | First Algorithm Behavior | Second Algorithm Behavior |
|---|---|---|
| **Sort Comparison** | `sort()`: Fast $O(N \log N)$, relative order of equal elements not guaranteed. | `stable_sort()`: Preserves relative order of equal elements ($O(N \log^2 N)$). |
| **Search Comparison** | `find()`: Linear search $O(N)$ on any range. | `binary_search()`: Fast $O(\log N)$ check on sorted ranges. |

### Table 7: pair vs tuple

| Feature | `std::pair` | `std::tuple` |
|---|---|---|
| **Capacity** | Exactly two elements (`first`, `second`) | Arbitrary number of elements (1, 2, 3, ...) |
| **Access** | `.first`, `.second` | `std::get(t)` |

IT STUDY HUB

# Big-O Complexity Quick Reference

Knowing Big-O complexities is vital for technical interviews and high-performance system design.

| Container / Operation | Insertion | Deletion | Search / Access |
|---|---|---|---|
| **`vector`** | $O(1)$ amortized (end) / $O(N)$ (middle) | $O(1)$ (end) / $O(N)$ (middle) | $O(1)$ (`[]`) |
| **`list`** | $O(1)$ (anywhere with iterator) | $O(1)$ (anywhere with iterator) | $O(N)$ (linear traversal) |
| **`map` / `set`** | $O(\log N)$ | $O(\log N)$ | $O(\log N)$ |
| **`unordered_map`** | $O(1)$ average ($O(N)$ worst) | $O(1)$ average ($O(N)$ worst) | $O(1)$ average ($O(N)$ worst) |

# HANDS-ON PROJECT: ADVANCED INVENTORY MANAGEMENT SYSTEM

In this comprehensive hands-on project, we combine STL containers (`vector`, `unordered_map`, `priority_queue`), iterators, algorithms (`sort`, `find_if`), lambda expressions, pairs, and tuples into a professional inventory analytics suite.

## Project Specification

**Problem Statement:** Build an advanced Inventory Management System that stores products with IDs, names, prices, and stock quantities. The system must support adding products, searching by ID, sorting by price, filtering by stock threshold, and managing low-stock restocking alerts using a max-priority queue.

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <queue>
#include <tuple>
#include <algorithm>
#include <iomanip>
using namespace std;

// Product structure: ID, Name, Price, Quantity
struct Product {
    int id;
    string name;
    double price;
    int quantity;
};

class InventorySystem {
    private:
        unordered_map productMap; // O(1) lookup by ID
        vector productVector;          // For sorting & filtering
```

```
    public:
        void addProduct(int id, string name, double price, int qty)
{
            Product p = {id, name, price, qty};
            productMap[id] = p;
            productVector.push_back(p);
            cout << "Product '" << name << "' added successfully.
\n";
        }

        void searchById(int id) {
            auto it = productMap.find(id);
            if(it != productMap.end()) {
                cout << "Found -> ID: " << it->second.id
                     << " | Name: " << it->second.name
                     << " | Price: $" << it->second.price
                     << " | Qty: " << it->second.quantity << endl;
            } else {
                cout << "Product with ID " << id << " not found.\n";
            }
        }

        void sortByPrice() {
            sort(productVector.begin(), productVector.end(), []
(const Product &a, const Product &b) {
                return a.price < b.price;
            });
            cout << "\n--- Products Sorted by Price (Low to High)
---" << endl;
            for(const auto &p : productVector) {
                cout << p.name << " - $" << fixed << setprecision(2)
<< p.price << " (Qty: " << p.quantity << ")" << endl;
            }
        }

        void checkLowStock(int threshold) {
            // Priority queue ordering products by lowest quantity
first
            auto cmp = [](const Product &a, const Product &b) {
                return a.quantity > b.quantity; // Min-heap behavior
for quantity
            };
            priority_queue, decltype(cmp)> lowStockPQ(cmp);

            for(const auto &pair : productMap) {
                if(pair.second.quantity <= threshold) {
```

```
                    lowStockPQ.push(pair.second);
                }
            }

            cout << "\n--- Low Stock Alert (Threshold <= " <<
threshold << ") ---" << endl;
            if(lowStockPQ.empty()) {
                cout << "All items are sufficiently stocked.\n";
                return;
            }

            while(!lowStockPQ.empty()) {
                Product top = lowStockPQ.top();
                lowStockPQ.pop();
                cout << "[ALERT] " << top.name << " (ID: " << top.id
<< ") has only " << top.quantity << " units left!\n";
            }
        }

        tuple getInventorySummary() {
            double totalVal = 0;
            int totalUnits = 0;
            string richestItem = "None";
            double maxPrice = -1;

            for(const auto &p : productVector) {
                totalVal += (p.price * p.quantity);
                totalUnits += p.quantity;
                if(p.price > maxPrice) {
                    maxPrice = p.price;
                    richestItem = p.name;
                }
            }
            return make_tuple(totalVal, totalUnits, richestItem);
        }
};

int main() {
    InventorySystem inv;

    inv.addProduct(101, "Laptop", 1200.50, 5);
    inv.addProduct(102, "Mouse", 25.99, 45);
    inv.addProduct(103, "Monitor", 300.00, 3);
    inv.addProduct(104, "Keyboard", 75.00, 12);

    inv.searchById(102);
```

```
    inv.sortByPrice();
    inv.checkLowStock(10);

    // Unpacking tuple summary
    auto [totalVal, totalUnits, richestItem] =
inv.getInventorySummary();
    cout << "\n--- Inventory Analytics (Tuple Unpacked) ---" <<
endl;
    cout << "Total Inventory Value: $" << fixed << setprecision(2)
<< totalVal << endl;
    cout << "Total Item Units     : " << totalUnits << endl;
    cout << "Most Expensive Item  : " << richestItem << endl;

    return 0;
}
```

**Expected Output:**

```
Product 'Laptop' added successfully.
Product 'Mouse' added successfully.
Product 'Monitor' added successfully.
Product 'Keyboard' added successfully.
Found -> ID: 102 | Name: Mouse | Price: $25.99 | Qty: 45

--- Products Sorted by Price (Low to High) ---
Mouse - $25.99 (Qty: 45)
Keyboard - $75.00 (Qty: 12)
Monitor - $300.00 (Qty: 3)
Laptop - $1200.50 (Qty: 5)

--- Low Stock Alert (Threshold <= 10) ---
[ALERT] Monitor (ID: 103) has only 3 units left!
[ALERT] Laptop (ID: 101) has only 5 units left!

--- Inventory Analytics (Tuple Unpacked) ---
Total Inventory Value: $8384.55
Total Item Units     : 65
Most Expensive Item  : Laptop
```

**Step-by-Step Explanation:**

- `unordered_map` provides $O(1)$ product lookup by ID.
- `vector` maintains a linear buffer for sorting by price using `std::sort` with custom lambda comparators.
- `priority_queue` with a custom comparator (`decltype(cmp)`) constructs a min-heap to surface lowest stock items instantly.
- `std::tuple` and structured binding (`auto [...]`) return and unpack multi-variable analytics summary metrics cleanly.

> **Practice Challenges:**
>
> - Add product removal and stock update functionalities.
> - Implement category filtering using `std::copy_if` algorithm.

IT STUDY

# Module Assessments

Test your mastery of STL containers, iterators, algorithms, and generic programming with these rigorously curated assessment questions.

## 15+ Practice Questions

1. What is the difference between a sequence container (`vector`, `deque`, `list`) and an associative container (`set`, `map`)?
2. Why does `std::vector` provide faster random access than `std::list`?
3. Explain how Red-Black Trees maintain $O(\log N)$ complexity in `std::map`.
4. What is the difference between `std::map` and `std::unordered_map`?
5. How does a `priority_queue` differ from a standard `queue`?
6. What are the five main categories of iterators in the STL?
7. Write a program that uses `std::sort` to sort a vector of integers in descending order using a lambda expression.
8. Explain the purpose of `std::binary_search` and why the container must be sorted first.
9. What is the difference between `lower_bound` and `upper_bound`?
10. How do lambda functions capture local variables by value vs reference?
11. What is a functor (function object), and how does it compare to a lambda function?
12. Write a generic template function that returns the minimum of two values of any data type.
13. How do you unpack a `std::tuple` using modern C++ structured bindings?
14. What is the Big-O insertion and lookup complexity of `std::unordered_map` in the average vs worst case?
15. Write a program that uses `std::accumulate` to find the product of all elements in an integer vector.

## 20+ Viva Questions (With Answers)

**Q1: What is the Standard Template Library (STL)?**

> A library of generic container classes, iterators, and algorithms in C++.

**Q2: What are the three core pillars of STL?**

> Containers, Iterators, and Algorithms.

**Q3: What is a sequence container?**

> A container that stores elements in a linear sequence (e.g., `vector`, `deque`, `list`).

**Q4: What is an associative container?**

> A container that stores elements in sorted tree structures based on keys (e.g., `set`, `map`).

**Q5: What is an unordered container?**

> A hash table-based container providing average $O(1)$ lookups (e.g., `unordered_map`).

**Q6: What is a container adapter?**

> A specialized wrapper providing restricted interfaces over underlying sequence containers (e.g., `stack`, `queue`, `priority_queue`).

**Q7: What is an iterator?**

> A pointer-like object used to traverse container elements.

**Q8: What is a random-access iterator?**

> An iterator that supports constant-time jumping (`it + n`) in addition to increment/decrement.

**Q9: What does `std::sort` do?**

> Sorts an iterator range in $O(N \log N)$ time using introsort.

**Q10: What is the time complexity of `std::binary_search`?**

> $O(\log N)$, requiring a sorted container.

**Q11: What does `lower_bound` return?**

> An iterator pointing to the first element that is not less than (i.e. greater than or equal to) a given value.

**Q12: What is a lambda function?**

> An anonymous inline function closure defined directly where it is called or passed.

**Q13: What is a functor?**

> An object of a class that overloads `operator()`, allowing it to be called like a function.

**Q14: What is a template in C++?**

> A feature enabling generic programming for functions and classes independent of specific data types.

**Q15: What is `std::pair`?**

> A utility structure holding two heterogeneous values (`first` and `second`).

**Q16: What is `std::tuple`?**

> A fixed-size collection holding an arbitrary number of heterogeneous values.

**Q17: What is cache locality in `std::vector`?**

> Contiguous RAM storage that allows CPU hardware caches to prefetch elements rapidly.

**Q18: What is the difference between `map` and `multimap`?**

> `map` requires unique keys; `multimap` permits duplicate keys.

**Q19: What is a priority queue?**

> A container adapter acting as a heap where the highest-priority element is always at the top.

**Q20: Why are STL algorithms separate from containers?**

> To decouple data structures from processing logic, allowing any algorithm to operate on any container via iterators.

## 15+ Technical Interview Questions

1. **Explain how `std::vector` grows its internal capacity under the hood.** Discuss capacity doubling, amortized $O(1)$ complexity, and memory reallocations.
2. **What is the difference between `std::map` and `std::unordered_map` regarding collision handling and memory overhead?** Compare Red-Black Trees vs Hash tables with bucket chains.
3. **When would you choose `std::list` over `std::vector`?** Discuss scenarios requiring frequent $O(1)$ middle insertions and deletions without random access needs.
4. **How do lambda captures work? Explain `[=]`, `[&]`, and explicit captures.** Detail stack frame lifetimes and dangling reference hazards.
5. **What are iterator invalidation rules for `std::vector` vs `std::list`?** Explain how vector reallocations invalidate all iterators, whereas list node deletions only invalidate iterators pointing to erased nodes.
6. **How do you implement a custom comparator for `std::priority_queue` to create a min-heap?** Provide syntax using custom structs or `std::greater`.
7. **What is template metaprogramming basics in STL (`std::enable_if`, `type_traits`)? (Preview)** Discuss compile-time type introspection.
8. **Explain how `std::lower_bound` works and its time complexity.** Detail binary search partitioning on sorted ranges.
9. **What is the purpose of `std::forward_list` compared to `std::list`?** Discuss single-linked memory savings where backward traversal is unnecessary.
10. **How do function objects (functors) outperform lambda functions in certain template contexts?** Explain inlining and stateful optimization.
11. **What is the time complexity of searching in an `unordered_map` when a worst-case hash collision occurs?** Explain $O(N)$ degradation.
12. **How does `std::deque` maintain $O(1)$ random access across chunked memory blocks?** Discuss map pointers to fixed-size array buffers.
13. **What is generic programming, and how does template instantiation happen at compile time?** Detail code bloat considerations and code generation.
14. **How do you remove elements from a `std::vector` while iterating through it safely? (Erase-remove idiom)** Explain `v.erase(remove_if(...), v.end())`.

15. **What is the difference between `std::set` and `std::unordered_set`?** Compare ordered Red-Black trees vs hash set lookups.

IT STUDY HUB

# Module 7 Wrap-Up

## Key Terms Glossary

- **STL:** Standard Template Library containing containers, iterators, and algorithms.
- **Sequence Container:** Linear data structures (`vector`, `deque`, `list`).
- **Associative Container:** Tree-based sorted collections (`set`, `map`).
- **Unordered Container:** Hash table-based collections (`unordered_map`).
- **Iterator:** Pointer-like object traversing container ranges.
- **Lambda:** Anonymous inline function closure.
- **Template:** Generic programming construct for type-independent code.

## Quick Revision Summary

Module 7 masters the Standard Template Library and generic programming. Sequence containers (`vector`, `deque`, `list`) manage linear data. Associative (`set`, `map`) and unordered containers (`unordered_map`) provide ordered and hash-indexed lookups. Container adapters (`stack`, `queue`, `priority_queue`) enforce specialized behaviors. Iterators bridge containers and algorithms (`sort`, `find`, `binary_search`). Lambda functions, functors, pairs, tuples, and templates empower robust generic C++ engineering.

## STL Coding Challenges

Complete these challenges before moving to Module 8:

- **Challenge 1: Frequency Word Counter.T** Read a large text document into a `std::vector`, sort it, and use `std::map` to compute and print word frequencies in alphabetical order.
- **Challenge 2: Custom Functor Filter.T** Write a custom functor class that filters numbers from a vector based on user-defined divisibility rules.
- **Challenge 3: Generic Sorter Template.T** Design a generic template function that sorts any STL sequence container regardless of its underlying data type.

## CONGRATULATIONS ON COMPLETING MODULE 7!

WHAT COMES NEXT:

**MODULE 8 — ADVANCED OOP, TEMPLATES & MODERN C++**